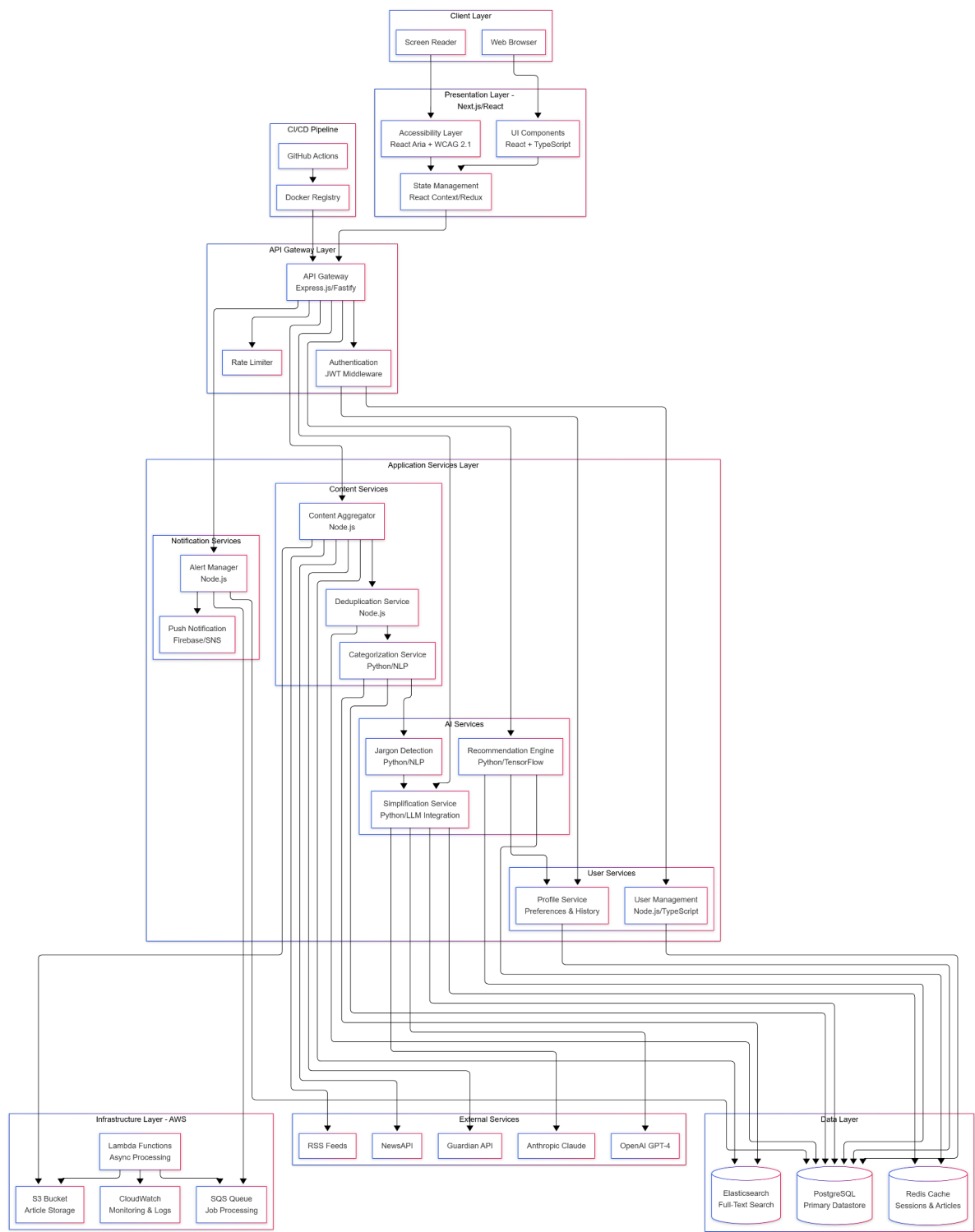


News For U High-Level Architecture

1.1 Architecture Diagram



1.2 Layer-by-Layer Architectural Rationale

Client Layer

- **Web Browser:** Primary interface for desktop and mobile users
- **Screen Reader Integration:** Direct compatibility layer ensuring WCAG 2.1 AA compliance through semantic HTML and ARIA attributes

Presentation Layer

- **Technology:** Next.js with React and TypeScript
- **Rationale:** Server-side rendering improves SEO and initial load performance; TypeScript ensures type safety
- **Accessibility Layer:** React Aria provides comprehensive screen reader support and keyboard navigation
- **Design Decisions:**
 - Component-based architecture enables reusable accessibility patterns
 - State management handles user preferences for text size, contrast modes, and complexity levels
 - Tailwind CSS core utilities ensure responsive design without compilation overhead

API Gateway Layer

- **Technology:** Express.js or Fastify (Node.js)
- **Responsibilities:**
 - Request routing and load balancing
 - JWT-based authentication and authorization
 - Rate limiting (100 requests/hour per user for LLM-intensive operations)
 - Request/response transformation
- **Interface Contracts:**
 - RESTful API design with JSON payloads
 - Standardized error responses (HTTP status codes + error objects)
 - Authentication: Bearer token in Authorization header
 - Versioning: `/api/v1/` prefix for future compatibility

Application Services Layer

User Services

- **User Management:** Handles registration, login, password reset, account deletion (Stories 1.1-1.4)
- **Profile Service:** Manages user preferences, reading history, topic subscriptions
- **Fault Tolerance:** Graceful degradation—personalization features degrade to defaults if profile service fails

Content Services

- **Content Aggregator:** Fetches articles from multiple sources every 15 minutes (Story 2.1)
 - Asynchronous processing via SQS queues
 - Retry logic with exponential backoff for failed API calls

- Circuit breaker pattern to prevent cascading failures
- **Deduplication Service:** Uses content hashing and fuzzy matching to identify duplicates (Story 2.2)
 - Threshold-based similarity detection (>85% similarity)
 - Preserves most complete version with earliest timestamp
- **Categorization Service:** NLP-based topic classification (Story 2.3)
 - Pre-trained models (e.g., Hugging Face transformers)
 - Fallback to keyword-based classification if ML service unavailable

AI Services

- **Simplification Service:** Core LLM integration for adaptive content (Stories 3.1-3.2)
 - **Input:** Original article text, target complexity level, user reading history
 - **Output:** Simplified text, semantic similarity score, processing metadata
 - **LLM Fallback Strategy:**
 1. Primary: OpenAI GPT-4 (higher quality, higher cost)
 2. Secondary: Anthropic Claude (fallback if rate limited)
 3. Tertiary: Rule-based simplification (vocabulary substitution)
 - Caching strategy: Cache simplified versions for 24 hours to reduce API costs
 - Timeout: 15 seconds per article (<1500 words)
- **Jargon Detection:** Identifies technical terms for highlighting (Story 3.3)
 - Domain-specific lexicons (finance, politics, technology, health)
 - Context-aware term extraction
 - Generates tooltip definitions via LLM or knowledge base
- **Recommendation Engine:** Collaborative filtering + content-based recommendations (Stories 4.2-4.4)
 - TensorFlow Serving or AWS SageMaker for model inference
 - Features: user reading history, topic preferences, article metadata
 - Weekly model retraining with user feedback data

Notification Services

- **Alert Manager:** Handles breaking news detection and topic subscriptions (Stories 7.1-7.3)
- **Push Notification:** Firebase Cloud Messaging (web) or AWS SNS (email)
- **Deduplication:** Event tracking to prevent duplicate alerts for same story

Data Layer

PostgreSQL (Primary Datastore)

- **Purpose:** Persistent storage for users, articles, preferences, feedback
- **Normalization:** 3NF to reduce redundancy
- **Indexes:** B-tree on user_id, article_id; full-text search indexes on titles
- **Backup:** Automated daily backups with point-in-time recovery

Redis (Cache)

- **Purpose:** Session management, simplified article caching, API rate limiting
- **TTL Strategy:**

- Sessions: 30 minutes
- Simplified articles: 24 hours
- API rate limits: 1 hour sliding window
- **Eviction Policy:** LRU (Least Recently Used)

Elasticsearch (Search Index)

- **Purpose:** Full-text search, trending topics, date-range filtering (Stories 5.1-5.4)
- **Index Structure:** Articles indexed by title, snippet, category, publication date, source
- **Refresh Interval:** 30 seconds for near-real-time search
- **Consistency:** Eventual consistency acceptable (async indexing via message queue)

External Services

News Aggregation APIs

- **NewsAPI:** 500 requests/day (free tier) for general news
- **Guardian API:** 12 requests/second for diverse perspectives
- **RSS Feeds:** Open-source news providers for legal compliance
- **Error Handling:** Log failures, skip unavailable sources, notify admins if >50% sources fail

LLM Services

- **OpenAI GPT-4:** Primary simplification engine
- **Anthropic Claude:** Secondary fallback
- **Rate Limits:** Respect API limits; queue requests during peak usage
- **Cost Management:** Cache aggressively, use shorter context windows

Infrastructure Layer (AWS)

- **S3 Bucket:** Archival storage for raw article content (immutable)
- **Lambda Functions:** Async processing for article fetching, background indexing
- **SQS Queues:** Decouple synchronous API requests from long-running AI tasks
- **CloudWatch:** Centralized logging, metrics, and alerting
- **Docker:** Containerized services for portability and consistent environments
- **GitHub Actions:** CI/CD pipeline with automated testing and deployment

1.3 Data Consistency Strategy

Challenge: Maintaining consistency across PostgreSQL (source of truth), Redis (cache), and Elasticsearch (search index).

Solution:

1. **Write Path:** All writes go to PostgreSQL first
2. **Cache Invalidation:** On update/delete, purge relevant Redis keys

3. **Search Indexing:** Async updates to Elasticsearch via SQS queue
 - Retry failed indexing operations
 - Periodic reconciliation job (nightly) to detect/fix drift
 4. **Acceptable Staleness:** Search results may lag by 30-60 seconds (eventual consistency)
-

1.4 Scalability Considerations

- **Horizontal Scaling:** Stateless services (API Gateway, Application Services) can scale behind load balancer
- **Database Scaling:** PostgreSQL read replicas for analytics queries; Redis clustering for cache distribution
- **Cost Optimization:** LLM API calls are the primary cost driver—aggressive caching and batch processing reduce expenses
- **Performance:** Median simplification time ≤ 15 seconds meets success criteria (Story 3.2)